

xiaozhi-esp32-server

临床营养师 AI Agent 系统

项目技术文档

基于 ESP32 语音交互的慢病管理 AI 营养师平台

版本: 1.0 | 日期: 2026-05-11

目录

第 1 章 项目概述

第 2 章 系统架构

第 3 章 知识检索系统

第 4 章 知识摄入管线

第 5 章 记忆系统

第 6 章 健康档案系统

第 7 章 临床安全拦截

第 8 章 插件函数系统

第 9 章 数据库架构

第 10 章 配置与部署

第 1 章 项目概述

1.1 项目定位

xiaozhi-esp32-server 是一个面向慢病管理的 AI 临床营养师系统。系统通过 ESP32 硬件设备进行语音交互，结合多层知识检索、长期记忆、健康档案和临床安全拦截，为用户提供个性化的饮食与营养指导。

1.2 技术栈

层级	技术	说明
硬件	ESP32-S3	语音采集与播放终端
通信	WebSocket	全双工实时音频传输
ASR	FunASR	语音识别 (Paraformer)
LLM	ChatGLM / Qwen	大语言模型推理
TTS	EdgeTTS	文本转语音
VAD	SileroVAD	语音活动检测
知识库	SQLite + FTS5	Wiki / RAG / 结构化知识
记忆	mem0 + PowerMem	四层认知记忆系统
安全	规则引擎	药物-食物禁忌拦截
语言	Python 3.10+	aiohttp / asyncio 异步架构

1.3 核心能力

- 语音对话：实时语音识别 + LLM 推理 + 语音合成，支持多轮上下文
- 多知识库检索：Wiki (TF-IDF)、RAG (BM25+向量)、结构化知识 (SQLite) 三路并行
- 长期记忆：四层认知架构 (working / factual / episodic / semantic)，跨会话持久化
- 健康档案：结构化存储用户疾病、用药、过敏、血糖等健康数据
- 临床安全拦截：确定性规则引擎，药物-食物禁忌检查，防止有害建议
- 知识摄入：PDF 指南自动解析 → Wiki 页面 + RAG 分块 + 结构化提取

第 2 章 系统架构

2.1 整体架构

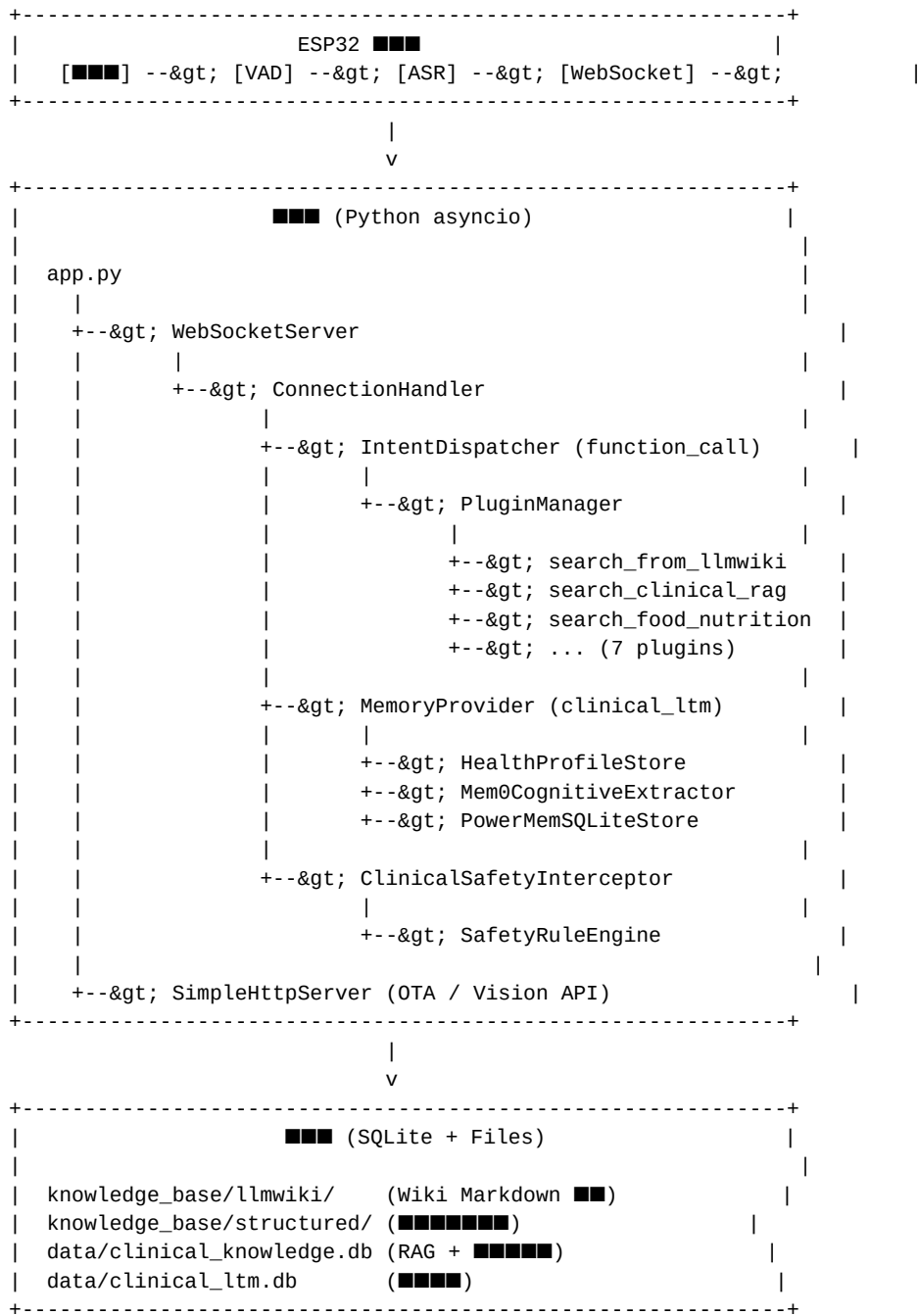


图 2-1: 系统整体架构图

2.2 请求处理流程

当 ESP32 设备发送语音数据时，处理流程如下：

1. VAD 检测语音活动，判断用户是否在说话
2. ASR (FunASR) 将语音转为文本

3. IntentDispatcher 判断是否触发插件函数 (function_call 模式)
4. 若触发插件: 调用对应工具函数, 获取知识上下文
5. ClinicalSafetyInterceptor 检查安全规则 (药物禁忌、过敏原)
6. MemoryProvider 注入记忆上下文 (working + long-term)
7. LLM 生成回答 (结合工具结果 + 记忆 + 健康档案)
8. TTS 将文本转为语音, 通过 WebSocket 返回设备

2.3 模块选择

各模块通过 config.yaml 的 selected_module 字段配置, 支持热插拔。当前默认配置: VAD=SileroVAD, ASR=FunASR, LLM=ChatGLMLLM, TTS=EdgeTTS, Memory=clinical_ltm, Intent=function_call。

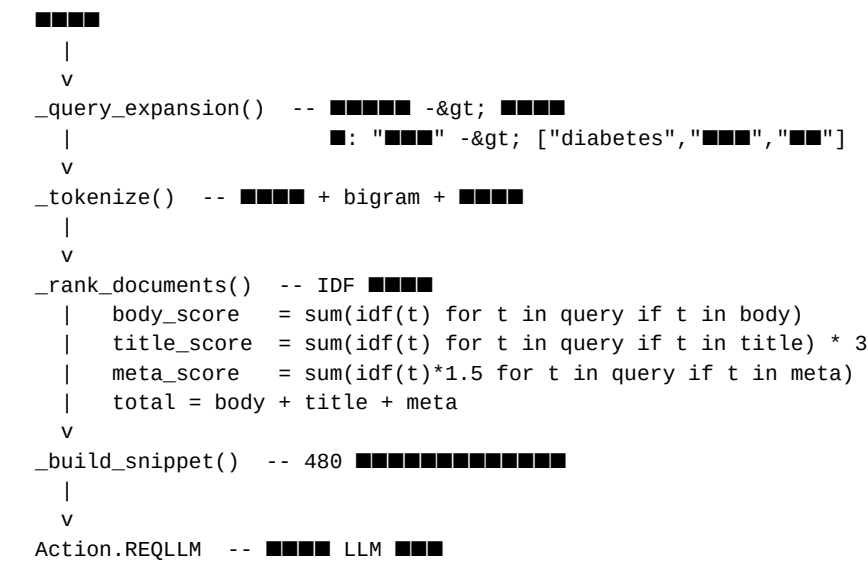
第 3 章 知识检索系统

系统采用三路并行检索架构，分别针对不同类型的临床营养知识：

3.1 Wiki 知识库 (TF-IDF 关键词检索)

Wiki 知识库以 Markdown 文件形式存储在 knowledge_base/llmwiki/clinical-nutrition/ 目录下。每个 Wiki 页面包含 frontmatter 元数据 (clinical_domain, conditions, kb_layer) 和正文内容。

检索流程：



关键实现文件：plugins_func/functions/search_from_llmwiki.py

参数	默认值	说明
top_k	4	返回最多 4 条结果
snippet_chars	480	每条结果截取的字符数
title 权重	3x	标题匹配得分为正文的 3 倍
meta 权重	1.5x	frontmatter 元数据匹配加权

3.2 RAG 混合检索 (BM25 + 向量)

RAG 检索由 ClinicalRAGService 实现，采用 BM25 词法检索与向量语义检索的混合策略。向量使用 DashScope text-embedding-v4 模型生成 (256 维)，存储为 SQLite BLOB，查询时全表扫描 + Python cosine similarity 计算。

混合评分公式：

```
total_score = 0.55 * bm25_score + 0.45 * cosine_similarity

bm25_score -- FTS5 BM25
```

```
cosine_score -- ████████████████████  
mmr_select() -- Maximal Marginal Relevance ████████
```

分块策略：

参数	值	说明
chunk_chars	850	每块最大字符数
chunk_overlap	150	相邻块重叠字符数
embedding_model	text-embedding-v4	DashScope 文本向量化模型
dimensions	256	向量维度

3.3 结构化知识库 (SQLite 关系表)

结构化知识由 StructuredKnowledgeStore 管理，存储在 clinical_knowledge.db 中。包含 15+ 张表，涵盖食物营养、食谱方案、食养方、MET 系数、诊断阈值等。

核心表：

表名	内容	查询函数
guide_tables / rows	指南表格数据	search_guide_tables()
food_exchange_portions	食物交换份	search_exchange_portions()
recipe_plans / meals / dishes	食谱方案	search_recipe_plans()
therapeutic_recipes	中医食养方	search_therapeutic_recipes()
activity_mets	运动 MET 系数	search_activity_mets()
diagnostic_thresholds	诊断阈值	search_diagnostic_thresholds()
nutrition_targets	营养目标	search_nutrition_targets()

3.4 食物营养数据库

食物营养数据存储在 knowledge_base/structured/clinical_nutrition.db 中，包含中国食物成分表、USDA 数据、血糖生成指数等。

表名	字段	说明
food_items	food_id, canonical_name, food_category	食物基础信息
food_aliases	food_id, alias, language	食物别名（中英文）
food_nutrients_per_100g	energy_kcal, protein_g, fat_g, ...	每 100g 营养成分
glycemic_values	gi_value, gl_value, serving_g	血糖生成指数

第 4 章 知识摄入管线

知识摄入管线 (Knowledge Ingestion Pipeline) 负责将 PDF 指南文档自动解析为 Wiki 页面、RAG 分块和结构化知识，是知识库的生产系统。

4.1 处理流程



图 4-1: 知识摄入管线流程

4.2 文档分析器 (DocumentProfiler)

DocumentProfiler 使用 LLM 分析 PDF 文档结构，识别 14 种内容块类型，并决定每页内容的存储路由。

块类型	说明	存储路由
narrative_guideline	叙述性指南内容	wiki + rag
diagnostic_threshold	诊断阈值数据	structured
nutrition_target	营养目标数据	structured
food_exchange_table	食物交换份表	structured
recipe_plan	食谱方案	structured
therapeutic_recipe	中医食养方	structured
met_coefficient	运动 MET 系数	structured

块类型	说明	存储路由
safety_contraindication	安全禁忌	structured + wiki
table_data	表格数据	structured
figure_legend	图表说明	rag
reference	参考文献	skip
toc	目录页	skip

4.3 Wiki 编译器 (Wiki Compiler v2)

Wiki Compiler 将文档按语义分块 (约 4200 字符/块), 每块由 LLM 生成: 核心结论 (key_conclusions)、行动建议 (clinical_recommendations)、安全边界 (safety_boundaries)、结构化候选 (structured_candidates)。

4.4 实际案例：成人肥胖指南

以《成人肥胖食养指南（2024年版）》（70 页）为例：

指标	数值
源文件页数	70 页
Wiki 页面数	7 个 (overview, principles, energy-control, food-selection, safe-weight-loss, regional-recipes, tcm-diet-therapy)
RAG 分块数	21 块
结构化提取	49 页内容
跳过页面	1 页 (目录页)
Wiki 覆盖率	69/70 页
RAG 覆盖率	70/70 页
LLM 审查状态	passed (confidence: 0.95)

关键文件：

文件	作用
core/clinical_nutrition/knowledge_ingestion.py	管线编排: create_draft() / approve_draft()
core/clinical_nutrition/document_profiler.py	LLM 文档结构分析
core/clinical_nutrition/wiki_compiler.py	Wiki 页面编译

文件	作用
data/knowledge_ingestion/drafts/	草稿输出目录

第 5 章 记忆系统 (Clinical LTM)

记忆系统采用四层认知架构，模拟人类记忆的形成、巩固和衰减过程，为 Agent 提供跨会话的用户理解能力。

5.1 四层认知架构

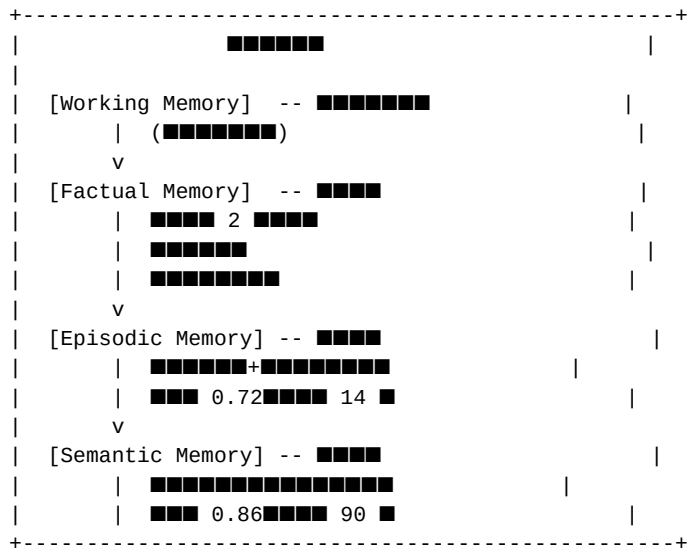


图 5-1: 四层认知记忆架构

5.2 MemoryProvider 编排流程

```
save_memory(messages, user_id, session_id):
  1. WorkingMemory -- ██████████ WorkingTurn
  2. HealthProfile -- ██████████
  3. ShortTermSummary -- ██████████ (██ working_memory_turns █████)
  4. Extraction -- mem0 hints + LLM ██████████
  |   +-- factual: ██████████
  |   +-- episodic: ██████████
  |   +-- semantic: ██████████
  5. Decay -- ██████████ importance/weight
  6. Upsert -- ██████████ SQLite (dedupe_key)
  7. PowerMem Sync -- ██████████ PowerMem ██████████
  8. Semantic Synthesis -- factual+episodic -> semantic ██████████
```

5.3 mem0 集成

mem0 作为语义预提取层，在 LLM 抽取之前提供 hints。支持两种模式：cloud (mem0 Cloud API) 和 local (mem0 本地 Qdrant 向量库)。当前配置为 local 模式，使用 Qdrant 内存向量库 + DashScope embedding。

5.4 记忆衰减机制

记忆类型	半衰期	衰减公式	说明
episodic	14 天	$w(t) = w_0 * 0.5^{(t/14)}$	事件记忆逐渐淡化
semantic	90 天	$w(t) = w_0 * 0.5^{(t/90)}$	规律记忆长期保留
factual	不衰减	weight 固定	锁定事实永远有效

5.5 语义合成

当同一实体的 factual + episodic 记忆积累到一定数量时，系统自动触发语义合成：LLM 将多条具体事件归纳为抽象规律。例如：多条「早餐吃高碳水食物导致血糖偏高」的 episodic 记忆，被合成为「用户早餐碳水偏好高，需控制份量」的 semantic 记忆。

5.6 关键文件

文件	作用
core/providers/memory/clinical_ltm/clinical_ltm.py	MemoryProvider 主编排
core/providers/memory/clinical_ltm/store.py	PowerMemSQLiteStore
core/providers/memory/clinical_ltm/extractor.py	Mem0CognitiveExtractor
core/providers/memory/clinical_ltm/lifecycle.py	MemoryLifecycleManager
core/providers/memory/clinical_ltm/interceptor.py	MemoryRetrievalInterceptor
core/providers/memory/clinical_ltm/models.py	Pydantic 数据模型
core/providers/memory/clinical_ltm/prompts.py	LLM 提取提示词

第 6 章 健康档案系统

HealthProfileStore
为每个用户维护结构化的健康数据，支持从对话中自动提取健康信息，并提供冲突检测和血糖追踪功能。

6.1 数据结构

数据类别	字段示例	存储表
基本信息	age, sex, height_cm, weight_kg, bmi	health_profiles
疾病	type-2-diabetes, hypertension	health_profile_items
用药	metformin, insulin	health_profile_items
过敏原	shrimp, peanut	health_profile_items
血糖记录	fasting=7.2, postprandial=11.5	blood_glucose_readings
回顾项	待确认的健康信息	health_profile_items (pending)

6.2 自动提取

每轮对话后，系统调用 `extract_health_profile_update()`
从用户消息中提取健康信息。提取逻辑包括：身高体重计算 BMI、血糖数值记录、疾病/用药/过敏原识别。
新增信息会与已有档案进行冲突检测：若用户之前说「对虾过敏」，后来又说「昨天吃了虾」，系统会标记为待回顾项。

6.3 血糖追踪

`blood_glucose_readings` 表记录用户的血糖测量值，包括：测量时间、血糖值 (mmol/L)、测量类型 (空腹/餐后/随机)。系统可自动分析血糖趋势，并在 Agent 回答中引用最近的血糖数据。

第 7 章 临床安全拦截

ClinicalSafetyInterceptor 是一个确定性规则引擎，在 LLM 生成回答之前检查潜在的临床安全风险。它不依赖 LLM，而是基于预定义的规则进行判定。

7.1 检查流程



图 7-1: 临床安全拦截流程

7.2 规则类型

规则类型	示例	处理方式
过敏原检查	用户对虾过敏 + 推荐了虾仁	BLOCK
药物-食物禁忌	服用华法林 + 推荐菠菜	BLOCK
疾病不适宜食物	痛风 + 推荐高嘌呤海鲜	WARN
自行调药风险	用户想减量二甲双胍	WARN + 安全提示

7.3 安全边界约束

Agent 在回答中必须遵守以下安全边界（通过系统提示词强约束）：

- 绝不建议用户自行调整药物剂量
- 绝不建议用户停药
- 食物建议必须注明「在医生指导下」
- 检测到严重症状时建议立即就医
- 不提供具体药物剂量建议
- 区分「知识库内容」和「通用推断」

第 8 章 插件函数系统

插件函数系统通过 `@register_function` 装饰器注册，由 `PluginManager` 统一管理。LLM 通过 `function_call` 机制调用插件，插件返回工具上下文供 LLM 生成回答。

8.1 注册机制

每个插件函数使用 `@register_function(name, description, tool_type)` 装饰器注册。函数签名统一为 `func(conn: ConnectionHandler, **params) -> ActionResponse`。 `ActionResponse` 包含三个字段： `action` (`REQLLM/RESPONSE`)、 `content`、 `error`。

8.2 已注册插件

插件名称	功能	输入参数	返回类型
search_from_llmwiki	Wiki 知识库检索	question: str	Action.REQLLM
search_clinical_rag	RAG 混合检索	question: str	Action.REQLLM
search_clinical_structured_knowledge	结构化知识检索	question: str	Action.REQLLM
search_food_nutrition	食物营养查询	food_name: str	Action.REQLLM
analyze_meal_nutrition	膳食营养分析	meal_description: str	Action.REQLLM
get_health_profile	读取健康档案	user_id: str	Action.RESPONSE
update_health_profile	更新健康档案	user_id, profile_data	Action.RESPONSE

8.3 调用流程



第 9 章 数据库架构

9.1 数据库总览

数据库文件	用途	主要表
clinical_knowledge.db	结构化知识 + RAG 分块 + 记忆	rag_documents, rag_chunks, rag_embeddings, guide_tables, recipe_plans, ...
clinical_ltm.db	长期记忆 + 工作记忆	ltm_working_memory, ltm_short_term_summary, ltm_memory_items
mem0_qdrant/	mem0 向量存储	Qdrant 内存向量库
clinical_nutrition.db	食物营养基础数据	food_items, food_nutrients_per_100g, glycemic_values, food_aliases

9.2 RAG 存储结构

```
rag_documents (■■■■)
  --- document_id, title, source_path, status, created_at

rag_pages (■■/■■)
  --- page_id, document_id, page_number, content, block_type

rag_chunks (■■)
  --- chunk_id, page_id, chunk_index, content, char_count

rag_embeddings (■■)
  --- chunk_id, embedding (BLOB), model_name, dimensions

rag_chunks_fts (FTS5 ■■■■)
  --- content (BM25 ■■)
```

9.3 记忆存储结构

```
ltm_working_memory (■■■■)
  --- user_id, session_id, turn_index, role, content, created_at

ltm_short_term_summary (■■■■)
  --- user_id, session_id, summary, turn_range, created_at

ltm_memory_items (■■■■)
  --- memory_id, user_id, layer (factual/episodic/semantic)
  --- entity, attribute, value, content, source
  --- importance, weight, dedupe_key
  --- observed_at, last_accessed, access_count
  --- tags, evidence (JSON), created_at
```

9.4 结构化知识存储

```
source_documents (■■■■)
```

guide_tables / guide_table_rows (■■■■)
food_exchange_portions (■■■■■)
recipe_plans / recipe_meals / recipe_dishes / recipe_ingredients (■■)
therapeutic_recipes (■■■■■)
activity_mets (■■ MET ■■)
diagnostic_thresholds (■■■■)
nutrition_targets (■■■■)
safety_rule_candidates (■■■■■)
needs_review (■■■■)

第 10 章 配置与部署

10.1 配置文件结构

系统使用 config.yaml 作为主配置文件。支持通过 data/.config.yaml 覆盖部分配置，保护密钥安全。

配置块	说明	关键字段
server	服务器基础配置	ip, port, websocket, http_port
selected_module	模块选择	VAD, ASR, LLM, TTS, Memory, Intent
plugins	插件配置	各插件的 API key、参数
clinical_rag	RAG 配置	chunk_chars, embedding, dimensions
clinical_ltm	记忆配置	working_memory_turns, mem0, powermem
llm	LLM 配置	provider, model, api_key, temperature
asr	ASR 配置	provider, model
tts	TTS 配置	provider, voice

10.2 LLM 配置

```
llm:
  type: openai # or dashscope, zhipuai, etc.
  # DashScope (Qwen)
  model: qwen-plus
  api_key: "your-api-key"
  base_url: "https://dashscope.aliyuncs.com/compatible-mode/v1"
  # [REDACTED] (0.0-1.0)
  temperature: 0.7
  # [REDACTED]
  max_tokens: 2000
```

10.3 RAG 配置

```
plugins:
  search_clinical_rag:
    chunk_chars: 850
    chunk_overlap: 150
    embedding:
      provider: dashscope
      model: text-embedding-v4
      dimensions: 256
    hybrid_weights:
      lexical: 0.55 # BM25 [REDACTED]
      vector: 0.45 # [REDACTED]
    top_k: 5
    mmr_lambda: 0.7 # MMR [REDACTED]
```

10.4 记忆系统配置

```
clinical_ltm:
  working_memory_turns: 12
  short_term_summary:
    enabled: true
    max_tokens: 500
  extraction:
    max_tokens: 1200
    temperature: 0.1
  lifecycle:
    episodic_half_life_days: 14
    semantic_half_life_days: 90
    synthesis_threshold: 5
mem0:
  enabled: true
  mode: local # or cloud
powermem:
  enabled: true
  vector_store:
    provider: qdrant
    path: data/mem0_qdrant
```

10.5 部署步骤

1. 安装依赖: `pip install -r requirements.txt`
2. 创建数据目录: `mkdir -p data/knowledge_base/llmwiki`
3. 复制配置模板: `cp config.yaml data/.config.yaml`
4. 编辑 `data/.config.yaml` 填入 API Key
5. 初始化数据库: `python -c "from core.clinical_nutrition.structured_knowledge import ..."`
6. 导入食物营养数据: `python scripts/import_china_food_composition_excel.py`
7. 摄入指南 PDF: `python scripts/ingest_hypertension_guide.py`
8. 启动服务: `python app.py`

10.6 环境依赖

依赖	版本	用途
Python	3.10+	运行时
aiohttp	3.9+	WebSocket / HTTP 服务
aiosqlite	0.19+	异步 SQLite
reportlab	4.4+	PDF 文档生成
pydantic	2.0+	数据模型
loguru	0.7+	日志
funasr	1.0+	语音识别
edge-tts	6.0+	文本转语音

依赖	版本	用途
mem0	0.1+	语义记忆
PyMuPDF	1.23+	PDF 文本提取